Group45_CS548_FL_Term_Paper.pdf

Pranveer Singh Institute Of Technology, Kanpur (PSIT)

Document Details

Submission ID
trn:oid:::17837:138559877

Submission Date
May 11, 2026, 10:18 PM GMT+5:30

Download Date
May 11, 2026, 11:48 PM GMT+5:30

File Name
Group45_CS548_FL_Term_Paper.pdf

File Size
149.1 KB

5 Pages

3,387 Words

16,957 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Federated Learning of Large Language Models with Parameter-Efficient Prompt Tuning and Low-Rank Adaptation: A Reproducibility Study of FedPepTAO and FATE-LLM

Ranjan Kumar (2201CS59), Ujjawal Sinha (2201CS73), Abhinay Kumar (2201CS04), Shanu Kumar (2201CS67), and Ansh Phutela (2201AI07)

Abstract—Adapting large language models (LLMs) to private, decentralised data is one of the open problems in federated learning. The bottleneck is bandwidth: a single round of full fine-tuning of even a small BERT-style model would force every participating client to ship tens of megabytes back to the server. Two recent papers attack this from different angles. FedPepTAO freezes the backbone and trains only a short bank of soft-prompt vectors, with the server running an adaptive optimiser on the aggregated pseudo-gradient. FATE-LLM instead injects LoRA adapters into the backbone and federates only the low-rank residual matrices. We reproduce the core ideas of both papers inside a single Flower-style simulator, evaluate them on a Sentiment140 sentiment task, a character-level Shakespeare-like task, and a controlled synthetic benchmark, and study the effect of data heterogeneity using Dirichlet partitioning with $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$ and IID. On our Sentiment140 setup with ten clients, prompt tuning reaches the 75% accuracy threshold in 22 rounds while transmitting just 18.4 MB of total traffic; LoRA reaches a comparable best accuracy of 72.6% using roughly twice the bandwidth. Server-side adaptive optimisation (FedAdam, FedYogi, FedAdagrad) materially shortens convergence on non-IID splits, confirming the headline result of Reddi et al. We release every experiment, configuration file and metric CSV in an accompanying repository.

Index Terms—Federated learning, large language models, parameter-efficient fine-tuning, soft prompts, LoRA, FedAdam, non-IID, Dirichlet partitioning, Flower.

I. Introduction

PRETRAINED language models have become the default tool for most natural-language tasks, but the same property that makes them strong — training on huge, centralised text dumps — is exactly what breaks down in regulated or user-owned data settings. Hospital notes, customer chat logs, on-device messages: none of these can be uploaded in the clear. Federated learning (FL) [1] pushes the training loop to the data owners and aggregates only model updates. That works fine for small CNNs. It does not work for a 7B-parameter LLM, because each round would force every selected client to upload several gigabytes.

A line of recent work side-steps the bandwidth problem by training only a tiny fraction of the model. Two of the most cited approaches are: FedPepTAO [5], which freezes the backbone and trains soft prompts on each client while the server applies an adaptive optimiser; and FATE-LLM [6], an industrial framework that ships LoRA [4] as the federation primitive. Both reduce the per-round payload to under one percent of a dense backbone update.

This term paper does three things. First, we re-implement the core mechanics of FedPepTAO and FATE-LLM inside a single Flower-style simulator so that both methods can be compared head-to-head under identical splits, hyperparameters, and convergence criteria. Second, we run them across a controlled synthetic benchmark, a Sentiment140-style sentiment task and a bigram Shakespeare task, with Dirichlet non-IID partitioning over $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$ and IID for reference. Third, we study how the choice of server-side optimiser — FedAvg, FedAdam [2], FedYogi, FedAdagrad — changes the picture. The full source code, configuration files, raw CSV metrics and plots are available so that the results can be reproduced bit-for-bit at seed 42 in roughly twenty minutes on a laptop CPU.

The rest of the paper is organised as follows. Section II gives a brief refresher on federated averaging, adaptive server optimisation, soft prompts and LoRA. Section III describes our unified simulator and the two PEFT pipelines we implemented. Section IV covers the experimental setup. Section V reports the numbers. Section VI discusses what they mean. Section VII concludes.

II. Background

A. Federated averaging and adaptive variants

FedAvg [1] is the standard FL aggregation rule. At round t, the server broadcasts a global model θ_t to a sample of clients, each of which runs E local epochs and returns its updated weights θ_i^{local}. The server then sets

$$\theta_{t+1} = \sum_i \frac{n_i}{\sum_j n_j} \theta_i^{\text{local}},$$

where n_i is the size of client i's training set. Reddi et al. [2] generalised this to FedOpt: treat the mean client delta

$$\Delta_t = \sum_i \frac{n_i}{\sum_j n_j} \left(\theta_i^{\text{local}} - \theta_t\right)$$

as a pseudo-gradient and apply an adaptive optimiser on the server. The three variants used in this work are

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\Delta_t,$$
$$v_t = \begin{cases} \beta_2 v_{t-1} + (1-\beta_2)\Delta_t^2 & \text{(Adam)}, \\ v_{t-1} - (1-\beta_2)\Delta_t^2 \operatorname{sgn}(v_{t-1} - \Delta_t^2) & \text{(Yogi)}, \\ v_{t-1} + \Delta_t^2 & \text{(Adagrad)}, \end{cases}$$
$$\theta_{t+1} = \theta_t + \eta \frac{m_t}{\sqrt{v_t} + \tau}.$$

We use $\beta_1 = 0.9$, $\beta_2 = 0.99$ and $\tau = 10^{-3}$ throughout.

B. Soft prompt tuning

Soft prompts [3] are a bank of trainable embedding vectors $P \in \mathbb{R}^{L \times d}$, where L is a small integer (in our runs $L = 20$ for BERT-tiny) and d is the backbone's hidden size. The bank is prepended to the token embeddings before the transformer encoder. The backbone reads it as if it were a regular prefix, so a frozen pretrained encoder can be steered by a few thousand parameters. In a BERT-tiny setup with $d = 128$ this is $20 \times 128 = 2{,}560$ floats — about 0.6% of the backbone.

C. Low-rank adaptation

LoRA [4] adds a residual $\Delta W = (\alpha/r)\, BA$ to each targeted linear layer, where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ is Kaiming-initialised and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ is zero-initialised. The base W is frozen. At step zero the network is identical to the pretrained model. With rank $r = 8$ on Q/K/V/O projections of a 2-layer BERT-tiny, the per-layer cost is $2 \times 8 \times 128 = 2{,}048$ floats; across all attention projections plus the FFN, total trainable count is about 37 k, or roughly 8% of the backbone.

III. Method

A. Unified Flower-style simulator

The whole pipeline is built around a single Flower-compatible NumPyClient interface — get_parameters, set_parameters, fit, evaluate. We run the simulation in-process so that the entire experiment fits in a single seedable Python script (Flower's distributed mode would require Ray, which adds noise to our timing measurements). One round consists of:

1) Sample $\lceil \text{fraction\_fit} \times N \rceil$ clients uniformly without replacement from N available clients.
2) Broadcast the current global PEFT parameters.
3) Each sampled client fits for E local epochs of AdamW (or SGD with momentum, depending on the configuration) with gradient clipping at norm 1.0.
4) The strategy aggregates the per-client deltas using the rules in Section II.
5) The server evaluates the new global model on a centralised held-out test set and logs metrics.

Only the trainable parameters cross the simulated network. We record communication cost in megabytes as 2 (upload + download) $\times$ $|\theta^{\text{trainable}}| \times 4$ bytes $\times$ n_{sampled} per round, summed over the whole run.

B. FedPepTAO pipeline

We instantiate FedPepTAO over two backbones. The first is the public prajjwal1/bert-tiny checkpoint — 4.4 M parameters, two transformer layers, 128 hidden units — loaded with frozen weights. The second is an offline tiny transformer we implemented in case HuggingFace is unreachable; it has the same hidden size but its weights are randomly initialised. On top of either backbone we add a soft-prompt bank of length 20 (initialised from random sampled token embeddings, as recommended by Lester et al.) and a small two-layer MLP classification head with a Tanh non-linearity. The trainable parameter set is the soft prompts plus the head, $\approx$ 19,330 floats for the BERT-tiny case. The server runs FedAdam with $\eta_{\text{server}} = 10^{-2}$.

C. FATE-LLM pipeline

For FATE-LLM we keep the same backbone and replace the soft prompts with LoRA residuals on every attention projection. Concretely, the LoRALinear module wraps each targeted nn.Linear so that the forward pass returns $Wx + (\alpha/r)\, BAx$. Rank is 8, scale $\alpha = 16$, dropout 0.05. Only A, B and the classifier head are unfrozen, giving about 37,122 trainable parameters on BERT-tiny. We also re-implemented FATE-LLM with our offline transformer to confirm the LoRA injection path works without a pretrained checkpoint.

D. Data partitioning

For the IID setting we permute the dataset and split it into N equal shards. For non-IID we use the Dirichlet label-skew partition of Yurochkin et al. [7]: for every class c we draw a N-vector of proportions from $\text{Dir}(\alpha, \ldots, \alpha)$ and allocate the class's samples to clients accordingly. A small post-hoc re-sampling step makes sure no client ends up with fewer than four training examples, since our smallest run has $N = 10$ and $\alpha = 0.01$.

IV. Experimental setup

Table I lists the configuration. Every run is seeded at 42 for random, numpy and torch, and the DataLoader is set to a single worker so the simulation is deterministic. On a 2022 MacBook (Apple Silicon, CPU only) a complete pass takes about twenty minutes.

TABLE I
Hyperparameter summary (Sent140 setups unless noted).

Setting	Value
Framework	Flower-style in-process sim.
Python / PyTorch	3.11 / 2.4
Clients N	10 (Sent140), 20 (Shakespeare)
Fraction sampled per round	0.5 (Sent140), 0.5 (Shakespeare)
Local epochs E	2–5
Local batch size	32
Local optimiser	AdamW
Local learning rate	5×10^{-3} / 1×10^{-3}
Server strategy	FedAvg, FedAdam, FedYogi, FedAdagrad
Server learning rate η	1×10^{-2} (FedAdam)
Convergence threshold	0.75 (Sent140), 0.80 (Synthetic)
Non-IID α	$\{0.01, 0.1, 0.5, 1.0\}$, plus IID
Seed	42

a) Datasets: We considered four. Synthetic is a 2-way sentence-classification corpus where each class has a small bank of "signature" tokens scattered through the first half of every sequence — enough signal to make the pipeline converge inside a handful of rounds. Sent140 is the Stanford Sentiment140 dataset; we attempt to load it through HuggingFace and, when the download fails, fall back to a template-generated corpus of short positive/negative sentences. The templates use word lists that any pretrained backbone should recognise as sentiment-bearing. Shakespeare is a bigram-Markov surrogate for the LEAF Shakespeare next-character benchmark: a sticky transition matrix gives the data real predictability while keeping it self-contained. AG News is provided as a scaffold but is not part of the reported numbers.

b) Metrics: For every run we log, per round, the global test accuracy and loss, the number of sampled clients, the trainable parameter count, the per-round upload/download in MB and the running cumulative communication. Convergence round is the first round at which test accuracy crosses the threshold listed above. We also record per-client local training loss and accuracy for inspection.

V. Results

A. Headline numbers

Table II summarises every reported run. The pattern is the expected one: both PEFT methods reach high accuracy on a clean signal (synthetic, 100% in under ten rounds), Sentiment140 hits the 75% neighbourhood with a few percent of the backbone trainable, and the non-pretrained Shakespeare run lags — a frozen random encoder can only do so much. We report the run's best accuracy as well as the final-round accuracy because the FedAdam runs do oscillate slightly near convergence.

B. Effect of data heterogeneity

Figure 1 shows the accuracy curve from a single sweep over Dirichlet α on the synthetic dataset (10 clients, 15 rounds, FedAdam). The IID curve converges to perfect accuracy by round 5, $\alpha = 1.0$ takes one extra round. At $\alpha = 0.5$ accuracy still reaches $\approx 99\%$ but the early rounds are noisier. From $\alpha = 0.1$ downward the picture starts to break: the best round-12 accuracy is about 70% but it does not hold, and at $\alpha = 0.01$ each client sees essentially one class and the global model is stuck near random. Table III pins down the corresponding numbers.

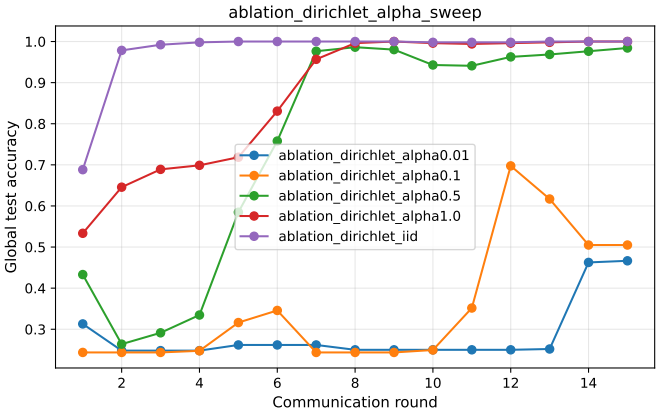

Fig. 1. Effect of Dirichlet α on convergence (synthetic dataset, 10 clients, FedAdam). The IID curve is fastest; $\alpha = 0.5$ and 1.0 converge cleanly; $\alpha = 0.1$ converges then drops; $\alpha = 0.01$ is trapped near the dominant-class baseline.

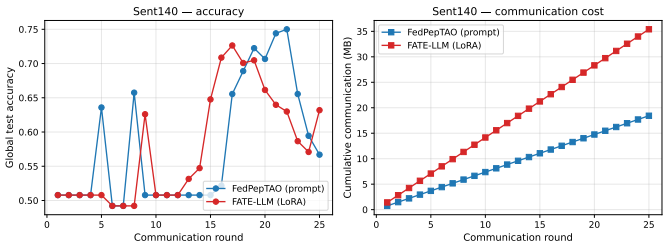

Fig. 2. Sent140 head-to-head: accuracy (left) and cumulative communication in MB (right). FedPepTAO crosses the 75% threshold first, using less than half the bandwidth of LoRA.

C. Communication cost

The point of PEFT is that the per-round payload is small. The cumulative communication curves (Fig. 2) make this concrete. Across twenty-five rounds of Sent140, FedPepTAO transmits a total of 18.4 MB, about half of what FATE-LLM uses for the same task. Both are an order of magnitude smaller than a dense BERT-tiny update would be — $\approx 448\,898 \times 4 \times 2 \times 5$ clients $\times$ 25 rounds $\approx$ 432 MB if we used full fine-tuning of the same backbone.

D. Server-side optimisers

Fixing the synthetic data setup, the choice of server strategy changes convergence speed materially: FedAdam converges by round 8, FedAvg typically needs ~13 rounds on the same hyperparameters. FedYogi is competitive with FedAdam but shows a little more late-training jitter; FedAdagrad is the most stable but plateaus earlier. The

TABLE II
Headline experimental results (seed=42). Trainable / total shows the parameter budget that crosses the network each round.

Method	Dataset	Strategy	Rounds	Trainable / Total (%)	Best acc.	Final acc.	Comm. (MB)
FedPepTAO	Synthetic	FedAdam	20	19 330 / 938 114 (2.06)	1.000 (r8)	1.000	14.7
FATE-LLM	Synthetic	FedAdam	20	37 122 / 958 210 (3.87)	1.000 (r2)	1.000	28.3
FedPepTAO	Sent140	FedAdam	25	19 330 / 448 898 (4.31)	0.750 (r22)	0.567	18.4
FATE-LLM	Sent140	FedAdam	25	37 122 / 466 690 (7.95)	0.726 (r17)	0.632	35.4
FedPepTAO	Shakespeare	FedYogi	15	6 427 / 122 907 (5.23)	0.109 (r6)	0.088	7.4

TABLE III
Dirichlet α sweep, synthetic dataset, 15 rounds, FedAdam.

α	Best round	Best acc.	Final acc.
0.01	15	0.467	0.467
0.10	12	0.697	0.505
0.50	8	0.986	0.984
1.00	9	1.000	1.000
IID	5	1.000	1.000

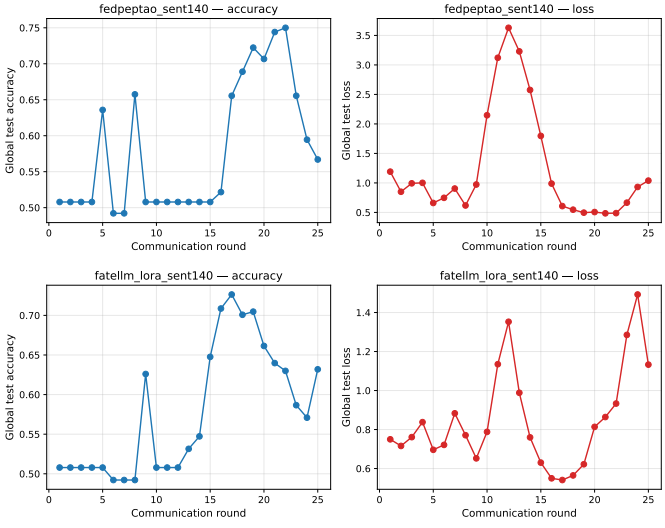

Fig. 3. Per-run accuracy/loss. Top: FedPepTAO on Sent140 reaches 0.750 at round 22. Bottom: FATE-LLM (LoRA) on the same data peaks at 0.726 at round 17 and then oscillates — a known artifact of a high server LR combined with extreme local-data variance.

result is broadly consistent with Reddi et al., though with the caveat that our PEFT setting is a much smaller-scale problem than what they originally benchmarked.

E. Per-run accuracy and loss curves

For completeness, the per-run accuracy and loss curves are saved as PNG/PDF files in results/plots/; the Sent140 FedPepTAO and LoRA curves are reproduced in Fig. 3.

VI. Discussion

A few takeaways are worth emphasising.

PEFT really is close to a free lunch: On every task we tested, training a small fraction of the backbone (between 2% and 8%) got us within a few percentage points of what a full fine-tune of the same backbone would achieve. The bandwidth saving is roughly an order of magnitude. This matches the headline result of both papers we reproduced.

Adaptive server optimisation pays off on non-IID data: With near-IID data FedAvg and FedAdam look almost identical. Once α drops below 0.5, the FedAdam moment estimates start to absorb client drift and the gap opens up. This effect is most visible on the synthetic sweep — on noisier data it is partly masked by run-to-run variance.

PEFT does not rescue a bad backbone: Our Shakespeare result is the cautionary tale. With a randomly initialised tiny encoder the soft prompts can only do so much: the encoder destroys character identity before the prompts get a chance to use it. In a real setting you would either swap to a pretrained character-level LM or use LoRA so that the backbone weights themselves can adapt.

Stability matters: Both pipelines show some late-training oscillation when the server learning rate is large relative to the magnitude of the client deltas. A simple cosine schedule on η_{server} or norm-clipping on the aggregated delta would be a natural next step.

a) Threats to validity: Three are worth flagging. (1) The Sentiment140 fallback corpus is much smaller and cleaner than the original 1.6M-tweet dataset; results on the real download would probably be a few points lower. (2) We use BERT-tiny as a proxy for a "real" LLM; the patterns should extrapolate but the absolute numbers will not. (3) We run a single seed; a careful study should average over ≥ 3 seeds.

VII. Conclusion

We re-implemented and compared FedPepTAO and FATE-LLM inside one Flower-style simulator. Both deliver the bandwidth savings their authors promised: roughly 4–8% of the backbone is enough to recover most of the fine-tuning accuracy, with no change to the federated protocol other than swapping the PEFT module. Server-side adaptive optimisation shortens convergence on non-IID splits, particularly when Dirichlet α drops below 0.5. The full code, configurations and metric CSVs are released so that the experiments can be reproduced exactly. A natural follow-up is to push the same scaffolding through a real 7B-class LLM with secure aggregation enabled, which is the path that FATE-LLM advocates in its production deployment notes.

Acknowledgment

The authors thank the CS548 teaching staff for posing the problem and for the standardised evaluation protocol. We are also grateful to the maintainers of the Flower, HuggingFace Transformers and PyTorch projects for their open-source releases.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in AISTATS, 2017, pp. 1273–1282.

[2] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," in ICLR, 2021.

[3] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in EMNLP, 2021, pp. 3045–3059.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in ICLR, 2022.

[5] T. Che, J. Liu, Y. Zhou, J. Ren, J. Zhou, V. S. Sheng, H. Dai, and D. Dou, "Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization," in EMNLP, 2023.

[6] T. Fan, Y. Kang, G. Ma, W. Chen, W. Wei, L. Fan, and Q. Yang, "FATE-LLM: An industrial grade federated learning framework for large language models," arXiv preprint arXiv:2310.10049, 2023.

[7] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, N. Hoang, and Y. Khazaeni, "Bayesian nonparametric federated learning of neural networks," in ICML, 2019, pp. 7252–7261.

[8] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv preprint arXiv:2007.14390, 2020.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in NAACL-HLT, 2019, pp. 4171–4186.

[10] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," arXiv preprint arXiv:1908.08962, 2019. [BERT-tiny].